

Ατομική Εργασία 1 – ΕΣΠΧ / Παράλληλα και Διανεμημένα Συστήματα

Ονοματεπώνυμο: Παπαρσενίου Απόστολος

Αριθμός Μητρώου: 11034

1. Εισαγωγή & Μεθοδολογία

Σκοπός της παρούσας εργασίας ήταν η μετατροπή του κλασικού παραδείγματος Παραγωγού-Καταναλωτή (prod_cons.c) σε ένα πλήρως λειτουργικό Thread Pool. Η FIFO ουρά τροποποιήθηκε ώστε αντί για απλούς ακεραίους να αποθηκεύει «πακέτα εργασίας», δηλαδή δομές της μορφής struct workFunction. Οι συναρτήσεις αυτές αναλαμβάνουν τον υπολογισμό του ημιτόνου για δέκα τυχαίες γωνίες, προσομοιώνοντας έτσι ένα πραγματικό υπολογιστικό φορτίο.

Για τη στατιστική ανάλυση του χρόνου αναμονής που μεσολαβεί από τη στιγμή που ένας producer βάζει μια εργασία στην ουρά μέχρι τη στιγμή που ένας consumer την παραλαμβάνει (πριν την εκτελέσει), χρησιμοποιήθηκε η συνάρτηση gettimeofday(). Συγκεκριμένα, καταγράφεται το χρονικό στίγμα (timestamp) κατά την εισαγωγή και αφαιρείται από το χρονικό στίγμα κατά την εξαγωγή, με το αποτέλεσμα να υπολογίζεται σε μικροδευτερόλεπτα (μs).

2. Υπολογιστικό Σύστημα

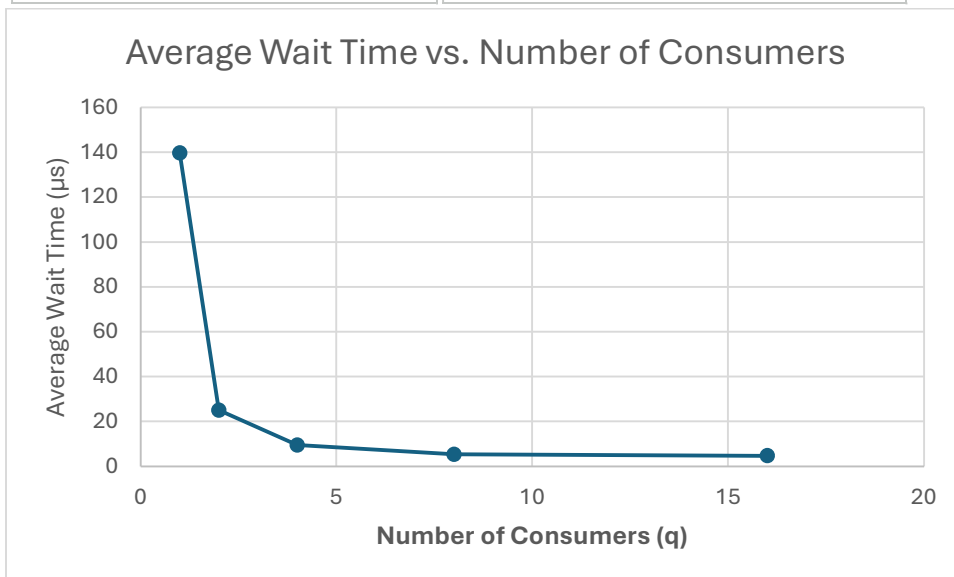
Τα πειράματα διεξήχθησαν σε υπολογιστικό σύστημα με Λειτουργικό Σύστημα Linux (μέσω Windows Subsystem for Linux - WSL). Ο επεξεργαστής του συστήματος διαθέτει **12 λογικούς πυρήνες** (σύμφωνα με την εντολή nproc). Ο πηγαίος κώδικας μεταγλωττίστηκε με τον GCC compiler κάνοντας χρήση της σημαίας μέγιστης βελτιστοποίησης -O3.

3. Πειραματικές Μετρήσεις

Για τον προσδιορισμό του βέλτιστου αριθμού καταναλωτών (consumers, q), πραγματοποιήθηκαν δοκιμές για $q \in \{1, 2, 4, 8, 16\}$. Ο αριθμός των παραγωγών (producers, p) διατηρήθηκε σταθερός στους 2, με τον καθένα να παράγει 10.000 εργασίες.

Για να εξασφαλιστεί η σταθεροποίηση των μετρήσεων και η αποφυγή τυχαίων διακυμάνσεων του λειτουργικού συστήματος, **κάθε πείραμα επαναλήφθηκε 20 φορές**. Στον παρακάτω πίνακα παρουσιάζεται ο συνολικός μέσος χρόνος αναμονής (σε μs) όπως προέκυψε από τον μέσο όρο των 20 εκτελέσεων:

Αριθμός Καταναλωτών (q)	Μέσος Χρόνος Αναμονής (μs)
1 Consumer	139.7275 μs
2 Consumers	25.0505 μs
4 Consumers	9.497 μs
8 Consumers	5.408 μs
16 Consumers	4.7135 μs



4. Συμπεράσματα

Από τα πειραματικά δεδομένα παρατηρούμε μια δραματική αρχική πτώση του χρόνου αναμονής καθώς αυξάνεται ο αριθμός των Consumers. Η μετάβαση από τον 1 στους 4 consumers μειώνει τον χρόνο αισθητά, επιβεβαιώνοντας τα πλεονεκτήματα του παραλληλισμού.

Ωστόσο, παρατηρούμε το φαινόμενο των φθινουσών αποδόσεων. Πλησιάζοντας τους 8 και 16 consumers, η μείωση του χρόνου ελαχιστοποιείται, δημιουργώντας ένα "πλατώ" (plateau).

Τελικό Συμπέρασμα: Ο αριθμός από νήματα consumer που ελαχιστοποιεί τη μέση τιμή του χρόνου αναμονής στο συγκεκριμένο σύστημα κυμαίνεται στους **8 με 10 consumers**. Η συμπεριφορά αυτή είναι απολύτως αναμενόμενη βάσει του hardware: διαθέτοντας 12 λογικούς πυρήνες και 2 νήματα Producer, απομένουν 10 φυσικοί πόροι για τους Consumers. Όταν το q γίνεται 16 (συνολικά 18 νήματα), ξεπερνάμε τους διαθέσιμους πυρήνες. Αυτό αναγκάζει το Λειτουργικό Σύστημα να κάνει συνεχές εναλλαγές (context switching) μεταξύ των νημάτων, εισάγοντας καθυστερήσεις και εμποδίζοντας την περαιτέρω βελτίωση του χρόνου.

5. Παράρτημα: Πηγαίος Κώδικας & Αναπαραγωγή Πειραμάτων

Ο πλήρης πηγαίος κώδικας του Thread Pool, καθώς και το αυτοματοποιημένο script (run_experiments.sh) που αναπτύχθηκε για τη διεξαγωγή των μετρήσεων, τη δυναμική αλλαγή των Consumers και την εξαγωγή του αρχείου results.csv, βρίσκονται αναρτημένα στο παρακάτω αποθετήριο στο GitHub:

- **Σύνδεσμος (Link):** <https://github.com/Tolizz/thread-pool-assignment>

*Αναλυτικές οδηγίες μεταγλώττισης και πλήρους αναπαραγωγής των πειραμάτων παρέχονται στο αρχείο **README.md** του αποθετηρίου.*